Module 2: Backend API

REST API Design Principles

REST = Representational State Transfer

Key principles:

- Resources identified by URLs (/users, /courses)
- Stateless: Each request contains all needed info
- Uniform interface: Consistent patterns

URL design:

- Nouns, not verbs: /users NOT /getUsers
- Plural names: /users NOT /user
- Hierarchical: /courses/{id}/sessions
- Query params for filtering: /users?role=student

HTTP Methods & Status Codes

HTTP Methods (CRUD operations):

- GET: Read (idempotent, cacheable)
- POST: Create (not idempotent)
- PUT: Full update (idempotent)
- PATCH: Partial update
- DELETE: Remove (idempotent)

Key Status Codes:

- 200 OK, 201 Created, 204 No Content
- 400 Bad Request, 401 Unauthorized, 403 Forbidden
- 404 Not Found, 422 Validation Error
- 429 Too Many Requests, 500 Server Error

JWT Authentication Flow

JWT = JSON Web Token

Structure: header.payload.signature

- Header: Algorithm (HS256)
- Payload: Claims (user_id, role, exp)
- Signature: HMAC(header + payload, secret)

Flow:

- 1. User logs in with credentials
- 2. Server validates, returns access + refresh tokens
- 3. Client sends: Authorization: Bearer <token>
- 4. Server verifies signature, extracts claims

Access token: Short-lived (1 hour)

Refresh token: Long-lived (7 days)

Database Schema Design Principles

Normalization: Eliminate redundancy

- Each fact stored once
- Use foreign keys for relationships

Primary keys:

- UUID preferred for distributed systems
- Auto-increment for simpler cases

Indexes: Speed up queries

- Index columns used in WHERE, JOIN, ORDER BY
- Don't over-index (slows writes)

Soft delete vs hard delete

- is_active = false vs DELETE
- Soft delete preserves audit trail

Rate Limiting & Security

Why rate limit?

- Prevent brute force attacks
- Protect against DoS
- Fair resource usage

Implementation with Redis:

- Key: rate_limit:{identifier}:{window}
- Increment counter on each request
- Set TTL = window duration

Common limits:

- Login: 60/hour per IP
- API: 1000/hour per user

Response: 429 Too Many Requests

Error Handling Patterns

Consistent error response format:

- { 'detail': 'Error message', 'code': 'ERROR_CODE' }

Don't leak internal details:

- Bad: 'PostgreSQL error at line 42'
- Good: 'Database error occurred'

Validation errors (422):

- List all invalid fields at once
- Include field name and reason

Log errors server-side

- Full stack trace for debugging
- Request ID for correlation